First, let’s clarify the rules of the game: given a target size for the tree, n, then we add (to both containers) the same n integers, chosen at random, without any limit; after these insertions, we perform another n operations: each of them can be removing an existing key (randomly chosen), or adding a new random integer to it. Obviously, we repeat this test for growing sizes.

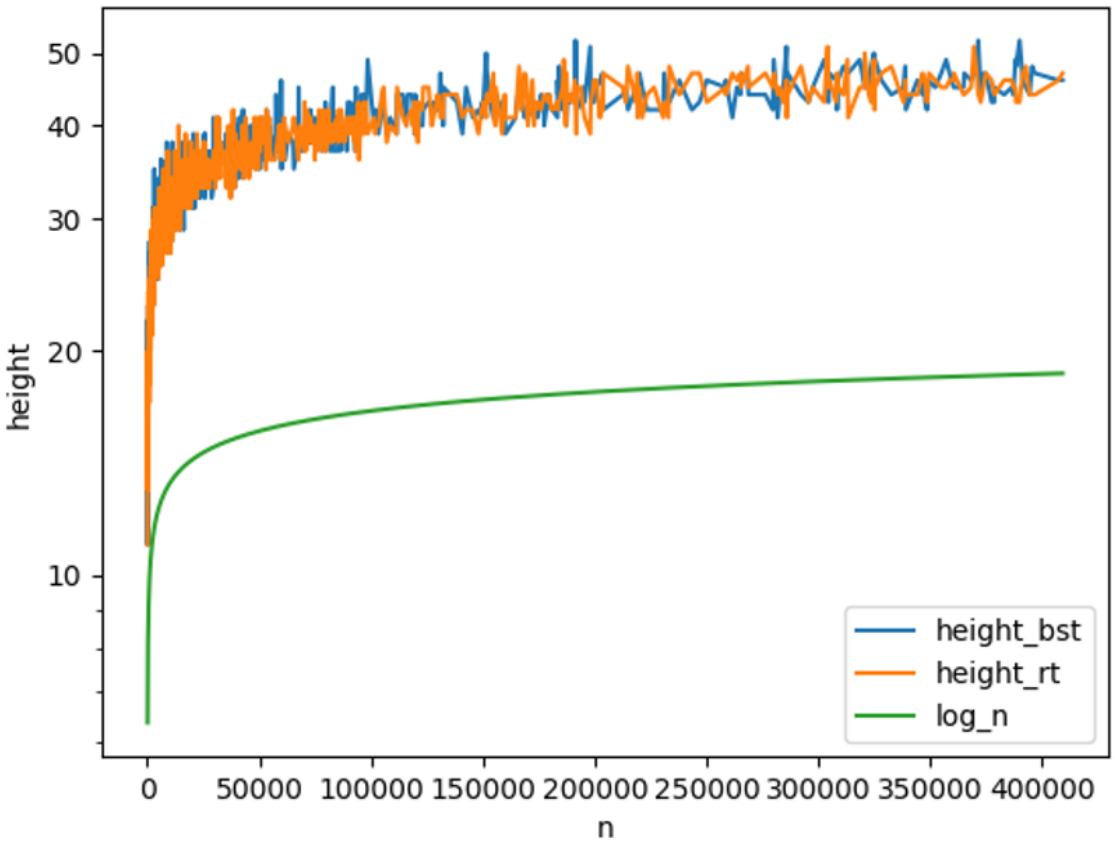


Figure 3.17 The height of a random treap versus a BST. On both data structures the same operations were performed, and the keys are random integers. Notice that the y axis uses a logarithmic scale.

We used an efficient implementation of BSTs (you can find it on the book’s repo) that limits the skewing effect of removing elements[13]; this expedient already improves the balance of BSTs, reducing the gap with balanced trees.